Evaluación Parcial Práctico

Algoritmos Iterativos vs Recursivos

Análisis Experimental de Complejidad Big-O

Estudiante: Henry Alonzo

Curso: Estructura de Datos

Catedrático: Ing. Brandon Chitay

Facultad: Ingeniería, Industria y Tecnología

Lenguaje: Java 11+

Fecha: Febrero 2026

Video de Demostración:	Video_ED_Parcial.webm

Introducción

¿Por qué importa Big-O?

En programación no basta con que algo funcione. También importa qué tan bien funciona cuando los datos crecen.

La notación Big-O no mide segundos exactos, sino cómo crece un algoritmo cuando el tamaño de entrada aumenta. Es una forma de predecir el comportamiento futuro del código antes de que el problema se vuelva grande.

En este proyecto no solo implementé algoritmos, sino que medí su rendimiento real para comprobar si su comportamiento coincide con la teoría. La intención fue observar cómo cambia el tiempo cuando n aumenta y validar experimentalmente su complejidad.

Más que memorizar fórmulas, el objetivo fue entender el crecimiento.

Descripción de los Algoritmos

Factorial

El factorial es un problema sencillo pero ideal para comparar iteración y recursión. En la versión iterativa, simplemente se multiplica desde 2 hasta n acumulando el resultado. En la versión recursiva, el problema se divide llamándose a sí mismo con n-1 hasta llegar al caso base. Ambas versiones realizan exactamente n operaciones, por lo que teóricamente son O(n). En este caso la diferencia entre iterativo y recursivo no es significativa en rendimiento.

Fibonacci

La serie de Fibonacci calcula el n-ésimo término donde cada número es la suma de los dos anteriores. Aquí es donde realmente se nota la diferencia entre enfoques. La versión iterativa guarda solo los dos valores anteriores y avanza paso a paso. Es eficiente y lineal. La versión recursiva sin memorización, en cambio, recalcula los mismos valores muchas veces. Cada llamada genera dos nuevas llamadas, formando un árbol que crece exponencialmente. Este fue el caso más interesante del proyecto porque la diferencia en tiempos fue muy evidente en las gráficas.

Búsqueda Lineal

Consiste en recorrer un arreglo elemento por elemento hasta encontrar el valor buscado o llegar al final. Tanto la versión iterativa como la recursiva realizan, en el peor caso, un recorrido completo del arreglo. Por eso ambas son O(n). Aquí el objetivo fue observar si la recursión introducía alguna diferencia perceptible en el tiempo.

Ordenamiento Burbuja

Burbuja compara elementos adyacentes e intercambia cuando están desordenados. Repite este proceso hasta ordenar completamente el arreglo. Su comportamiento es cuadrático porque para cada elemento se realizan múltiples comparaciones. Matemáticamente esto se aproxima a n^2 comparaciones. En este caso, tanto la versión iterativa como la recursiva mantienen el mismo orden de complejidad.

Fragmentos de Código Relevantes

Medición de Tiempos – Medidor.java

Medidor.java

```java
public class Medidor {
    public static double[] medir(Runnable algoritmo) {
        double[] tiempos = new double[REPETICIONES];
        for (int i = 0; i < REPETICIONES; i++) {
            long inicio = System.nanoTime();
            algoritmo.run();
            long fin = System.nanoTime();
            tiempos[i] = (fin - inicio) / 1_000_000.0;
        }
        return tiempos;
    }

```

Factorial Iterativo vs Recursivo

Factorial.java

```java
public class Factorial {
    // --- ITERATIVO ---
    public static long iterativo(int n) {
        long resultado = 1;
        for (int i = 2; i <= n; i++) {
            resultado *= i;
        }
        return resultado;
    }

    // --- RECURSIVO ---
    public static long recursivo(int n) {
        if (n <= 1) return 1;
        return n * recursivo(n - 1);
    }
}
```

Fibonacci Iterativo vs Recursivo

```java
Fibonacci.java
public class Fibonacci {  3 usages  Henry *
    // --- ITERATIVO ---
    public static long iterativo(int n) {  1 usage  Henry
        if (n <= 1) return n;
        long anterior = 0, actual = 1;
        for (int i = 2; i <= n; i++) {
            long siguiente = anterior + actual;
            anterior = actual;
            actual = siguiente;
        }
        return actual;
    }

    // --- RECURSIVO ---
    // Sin memorizacion: cada llamada genera dos llamadas nuevas → arbol exponencial
    public static long recursivo(int n) {  3 usages  Henry
        if (n <= 1) return n;
        return recursivo( n: n - 1) + recursivo( n: n - 2);
    }
}
```

Burbuja Recursivo

```java
OrdenamientoBurbuja.java
public class OrdenamientoBurbuja {  3 usages  Henry
    // --- RECURSIVO ---
    // Cada llamada "burbujea" el mayor elemento al final, luego reduce el problema en 1
    public static int[] recursivo(int[] arreglo) {  1 usage  Henry
        int[] arr = Arrays.copyOf(arreglo, arreglo.length);
        burbujearRecursivo(arr, arr.length);
        return arr;
    }

    private static void burbujearRecursivo(int[] arr, int n) {  2 usages  Henry
        if (n <= 1) return;
        for (int i = 0; i < n - 1; i++) {
            if (arr[i] > arr[i + 1]) {
                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
            }
        }
        burbujearRecursivo(arr, n: n - 1);
    }
}
```

Tabla de Tiempos Medidos

Factorial y Fibonacci

Algoritmo	Versión	n=5	n=10	n=15	n=20	n=25	n=30
Factorial	Iterativo	0.1328	0.0004	0.0005	0.0005	0.0007	0.0036
Factorial	Recursivo	0.0013	0.0006	0.0010	0.0008	0.0011	0.0068
Fibonacci	Iterativo	0.1085	0.0005	0.0005	0.0006	0.0007	0.0015
Fibonacci	Recursivo	0.0016	0.0355	0.0754	0.1800	0.5943	4.8181

Búsqueda Lineal y Burbuja

Algoritmo	Versión	n=100	n=500	n=1000	n=5000	n=10000
Búsqueda Lineal	Iterativo	0.1183	0.0075	0.0155	0.0758	0.2323
Búsqueda Lineal	Recursivo	0.0050	0.0788	0.0887	0.0718	0.0506
Burbuja	Iterativo	0.3069	1.5143	1.1532	27.762	134.76
Burbuja	Recursivo	0.0956	1.2127	3.4675	29.295	142.68

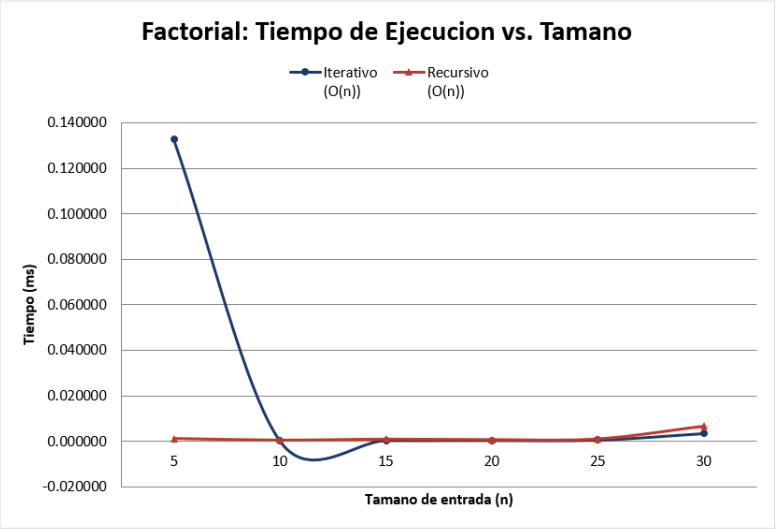
Gráficas de Rendimiento

Factorial: Iterativo vs Recursivo

HOJA — A1 - Factorial | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
5	0.132760	0.001340
10	0.000420	0.000600
15	0.000500	0.001040
20	0.000540	0.000820
25	0.000700	0.001140
30	0.003580	0.006800

Observacion: Ambas versiones son O(n): el tiempo crece linealmente con n.



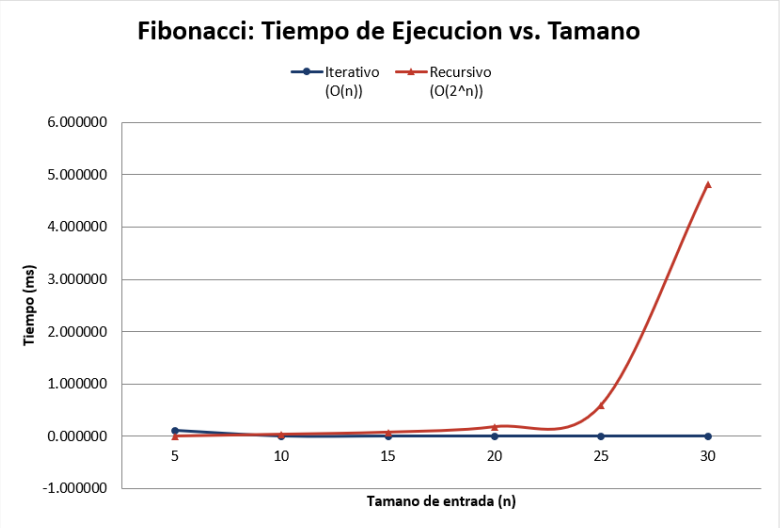
Ambas versiones muestran comportamiento O(n). El pico en n=5 del iterativo se debe al calentamiento de la JVM en la primera ejecución.

Fibonacci: Iterativo vs Recursivo

HOJA — A2 - Fibonacci | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(2^n))
5	0.108500	0.001580
10	0.000480	0.035540
15	0.000520	0.075400
20	0.000560	0.179960
25	0.000660	0.594280
30	0.001480	4.818141

Observacion: Recursivo crece exponencialmente: duplicar n casi cuadriplica el tiempo.



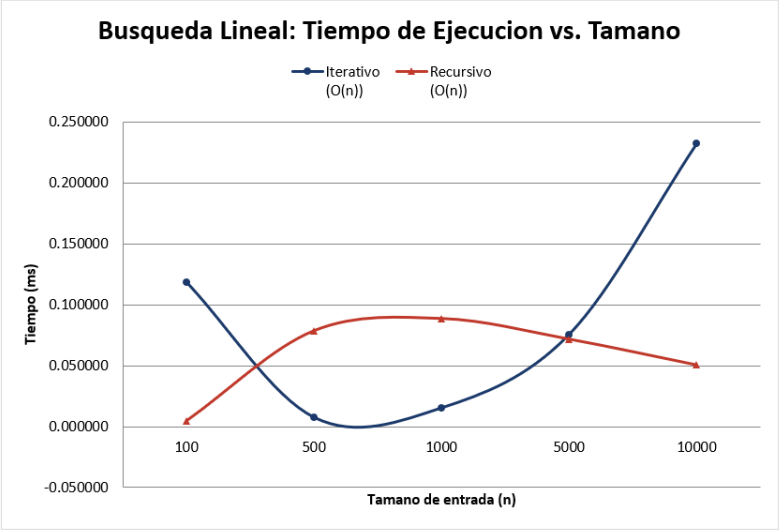
El recursivo explota exponencialmente — de 0.001ms en n=5 a 4.8ms en n=30. El iterativo permanece prácticamente plano.

Búsqueda Lineal: Iterativo vs Recursivo

HOJA — A3 - Busqueda Lineal | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
100	0.118340	0.004960
500	0.007520	0.078820
1000	0.015460	0.088720
5000	0.075820	0.071760
10000	0.232340	0.050560

Observacion: Ambas O(n): al multiplicar n x10, el tiempo tambien x10 aprox.



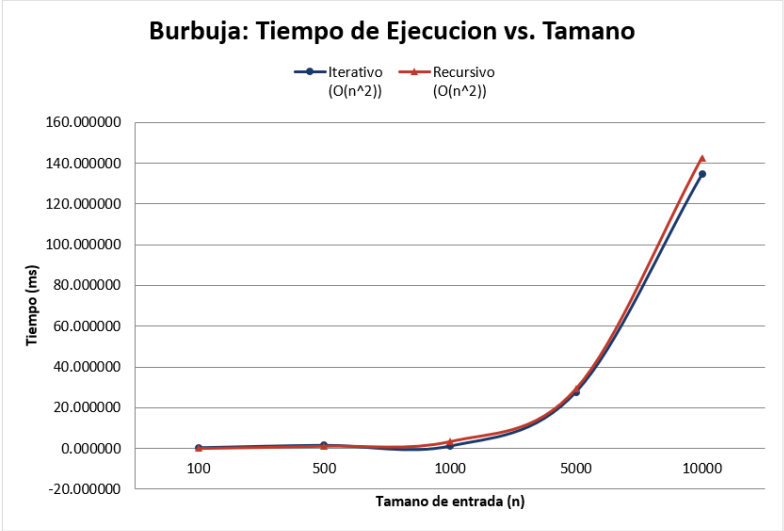
Ambas versiones son O(n). Las variaciones se deben al overhead de la pila de llamadas en el recursivo para n grandes.

Burbuja: Iterativo vs Recursivo

HOJA — A4 - Burbuja | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n^2)	Big-O Recursivo:

n	Iterativo (O(n^2))	Recursivo (O(n^2))
100	0.306940	0.095560
500	1.514260	1.212720
1000	1.153240	3.467460
5000	27.761680	29.294780
10000	134.756620	142.677480

Observacion: Ambas O(n^2): al x10 n, el tiempo aumenta aprox x100.



Curva cuadrática clara en ambas versiones. Al pasar de n=1000 a n=10000 (×10), el tiempo aumenta aproximadamente ×100, confirmando $O(n^2)$.

Análisis y Conclusiones

Al comparar las versiones iterativas y recursivas de cada algoritmo, se pudo observar que la diferencia no está en que una "funcione mejor" que otra, sino en cómo crece el número de operaciones cuando aumenta el tamaño de entrada.

En el caso de **Factorial**, ambas versiones mostraron un crecimiento lineal y tiempos muy similares. Esto confirma que cuando el número de operaciones es proporcional a n, la recursión no introduce una diferencia significativa más allá del uso de la pila de llamadas.

En **Búsqueda Lineal**, tanto la versión iterativa como la recursiva recorren el arreglo elemento por elemento en el peor caso. El comportamiento observado fue lineal, confirmando su complejidad O(n). Las pequeñas variaciones en tiempo se deben al costo adicional de las llamadas recursivas.

El caso más evidente fue **Fibonacci**. La versión iterativa mantuvo un crecimiento estable y controlado. En cambio, la versión recursiva mostró un aumento acelerado del tiempo conforme n crecía. Aquí se comprobó experimentalmente el comportamiento exponencial O(2^n). La diferencia entre ambas versiones fue considerable incluso en valores relativamente pequeños como n = 30.

En **Ordenamiento Burbuja**, tanto la versión iterativa como la recursiva mostraron una curva cuadrática clara. Cuando el tamaño del arreglo aumentó significativamente, el tiempo creció de manera mucho más pronunciada, confirmando su complejidad $O(n^2)$.

En general, los resultados experimentales coincidieron con la teoría de notación Big-O en todos los algoritmos evaluados.

Reflexión

Este proyecto me permitió observar de forma práctica algo que normalmente solo se ve en teoría: el crecimiento exponencial es extremadamente costoso. La versión recursiva de Fibonacci funciona correctamente, pero se vuelve impráctica rápidamente. Aunque conceptualmente es elegante, en términos reales no es eficiente sin optimizaciones como memorización. También pude notar que la recursión introduce un costo adicional en memoria debido al uso de la pila de llamadas. Para tamaños grandes, esto puede generar errores como StackOverflowError. En problemas simples como factorial, ambas versiones son prácticamente equivalentes. Sin embargo, en problemas donde el número de llamadas crece exponencialmente, la diferencia es crítica.

En aplicaciones reales, elegiría recursión cuando el problema sea naturalmente recursivo (como árboles o divide y vencerás), pero preferiría iteración cuando el rendimiento y la memoria sean factores importantes.

Respuestas Breves a las Preguntas Guía

¿Por qué la versión recursiva de Fibonacci es drásticamente más lenta que la iterativa a medida que n crece? Porque cada llamada genera dos llamadas adicionales y recalcula valores repetidos. Esto crea un árbol de ejecución exponencial, mientras que la versión iterativa solo realiza n pasos sin repetir cálculos.

¿Qué significa que un algoritmo sea $O(n^2)$? ,¿Cómo se refleja en la gráfica del Burbuja? Significa que el número de operaciones crece proporcionalmente al cuadrado del tamaño de entrada. En la gráfica de Burbuja se observa una curva parabólica: cuando n aumenta considerablemente, el tiempo crece de forma mucho más acelerada.

Si n se duplica, cuánto aumenta el tiempo de ejecución para O(n), $O(n^2)$ y O(2^n)?
Para O(n), el tiempo aproximadamente se duplica.
Para $O(n^2)$, el tiempo se cuadruplica.
Para O(2^n), el tiempo crece de manera exponencial, multiplicándose mucho más allá de un simple factor constante.

¿Cuál es el costo de usar recursión en Java en términos de memoria (stack)? Cada llamada recursiva ocupa un espacio en la pila de ejecución. Si el número de llamadas es muy grande, puede producirse un StackOverflowError. La versión iterativa no tiene este problema porque reutiliza el mismo espacio en memoria.

¿En qué situaciones elegirías la versión recursiva sobre la iterativa, y viceversa? Elegiría recursión cuando el problema sea naturalmente recursivo (árboles, divide y vencerás) y el código sea más claro de esa forma. Elegiría iteración cuando el rendimiento y el uso de memoria sean factores críticos.

Reflexión Final

Este proyecto permitió comprobar de forma práctica que la teoría de complejidad no es solo un concepto matemático, sino una herramienta real para predecir comportamiento y tomar decisiones de diseño. Entender cómo crece un algoritmo es tan importante como hacerlo funcionar correctamente.

Referencias

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

[2] Oracle Corporation. (2024). Java SE 11 Documentation. https://docs.oracle.com/en/java/javase/11/

[3] Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.

[4] GeeksforGeeks. (2024). Analysis of Algorithms — Big-O Notation. https://www.geeksforgeeks.org/analysis-of-algorithms-set-3asymptotic-notations/

[5] Chitay, B. (2026). Examen Parcial Práctico — Estructura de Datos. Universidad Da Vinci de Guatemala.

Anexo

Prompts de Inteligencia Artificial Utilizados

Prompt 1 — Planificación del Proyecto

"Analiza el documento oficial del examen parcial de Estructura de Datos y genera un plan estructurado de trabajo que divida el proyecto en fases lógicas, incluyendo organización del repositorio, implementación de algoritmos, medición de tiempos y documentación final."

Prompt 2 — Diseño de Arquitectura

"Proporciona una propuesta de estructura de carpetas y organización de clases en Java que cumpla con los requisitos del examen y permita una separación clara entre algoritmos, medición de rendimiento y generación de datos."

Prompt 3 — Implementación de Algoritmos

"Genera las implementaciones en Java de los algoritmos solicitados (Factorial, Fibonacci, Búsqueda Lineal y Ordenamiento Burbuja) en sus versiones iterativa y recursiva, siguiendo principios de clean code y comentarios mínimos pero claros."

Prompt 4 — Sistema de Medición

"Diseña una clase utilitaria en Java para medir tiempos de ejecución utilizando System.nanoTime(), ejecutando cada algoritmo múltiples veces y retornando los tiempos individuales y promedios en milisegundos."

Prompt 5 — Documentación Técnica

"A partir de los resultados experimentales obtenidos, redacta un documento técnico que incluya análisis comparativo entre versiones iterativas y recursivas, interpretación de gráficas y validación experimental de la complejidad Big-O."